Secure Internal App Delivery Platform on Azure

Technical project documentation covering architecture, implementation flow, tools used, and validation evidence

Prepared by

Mina Adel Zaki Gaballa

Documentation date: 2026-03-19

Document purpose

This report documents a full cloud platform implementation built with Terraform, AKS, ACR, Azure Arc, Flux GitOps, Azure Key Vault, Workload Identity, and Cluster Autoscaler. It explains what each tool does, why it was selected, and how the final platform was validated.

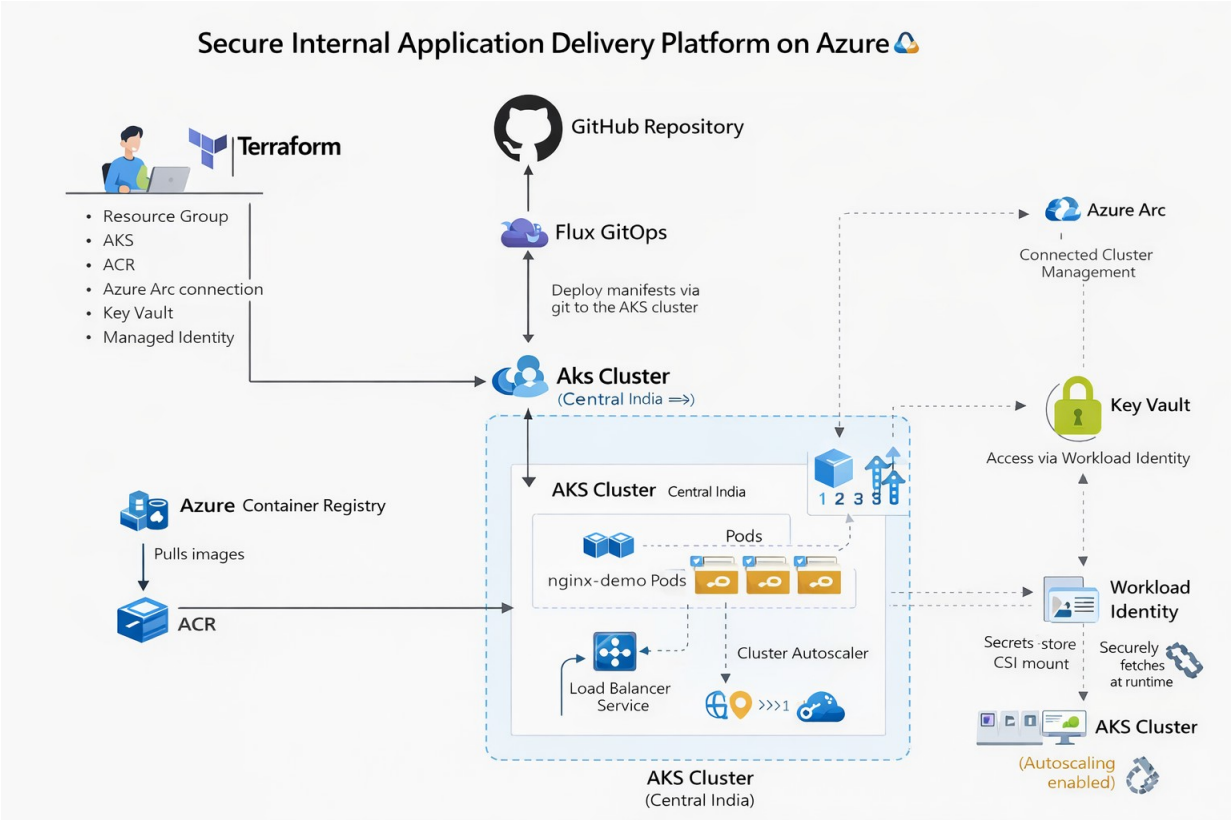

Figure 1. Final platform architecture diagram.

1. Executive Summary

This project was designed as a practical internal application delivery platform rather than a basic infrastructure lab. The implementation combined infrastructure provisioning, private container delivery, Git-driven deployment, centralized secret management, and hybrid-ready cluster management into one coherent Azure solution.

The final validated platform included Terraform for provisioning, Azure Kubernetes Service (AKS) for workload hosting, Azure Container Registry (ACR) for private image storage, Flux GitOps for deployment from GitHub, Azure Key Vault with Workload Identity for secret retrieval inside pods, Cluster Autoscaler for node elasticity, and Azure Arc for hybrid-ready cluster governance.

After validation, the Azure resource group was intentionally removed to avoid ongoing cloud cost. The repository, screenshots, diagram, and deployment artifacts remain as the implementation record.

2. Business Problem and Project Goal

Many internal applications are still delivered through manual commands, static configuration files, and ad-hoc secret handling. That approach creates inconsistent deployments, weak security controls, poor change tracking, and limited scalability.

The project goal was to build a secure internal application delivery platform that solves those issues through:

- Infrastructure as Code instead of manual portal builds
- A managed Kubernetes runtime for application hosting
- Private image storage and controlled image pulls
- GitOps-based deployment from source control
- Centralized secrets in Key Vault rather than in code or manifests
- Secretless workload authentication through Workload Identity
- Autoscaling for infrastructure elasticity
- Hybrid-ready cluster management through Azure Arc

3. Final Architecture Overview

The platform architecture centers on AKS as the runtime layer, with Terraform providing infrastructure provisioning, GitHub plus Flux delivering the desired application state, ACR supplying container images, and Key Vault providing secrets to workloads through Workload Identity and the Secrets Store CSI Driver.

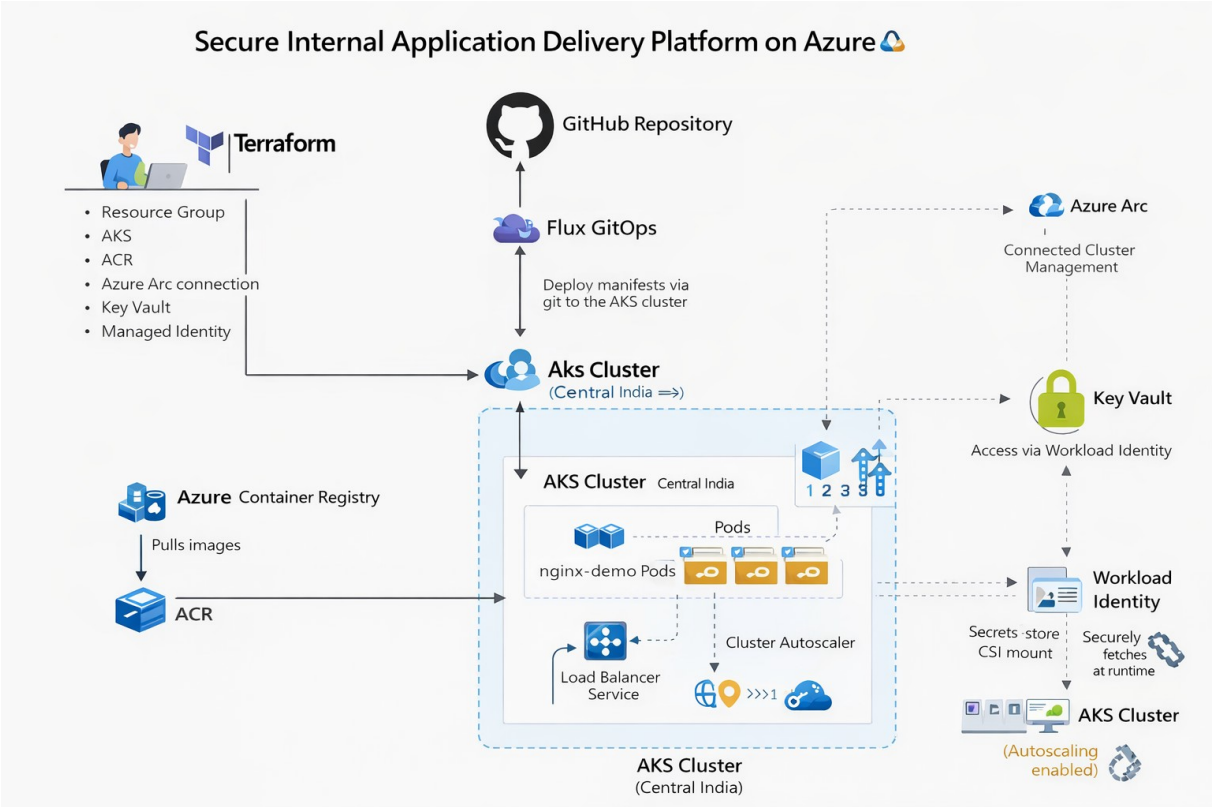

Figure 2. End-to-end platform architecture showing provisioning, delivery, security, and operations layers.

4. Tools Used and Why They Were Selected

The table below summarizes the main tools used in the project and the reason each one was included.

Tool / Service	What it does	Why it was used in this project
Terraform	Defines infrastructure as code	Used to provision repeatable Azure resources instead of building everything manually.
Azure CLI	Operational command-line interface for Azure	Used to authenticate, update AKS features, manage identities, Key Vault, Arc, and Flux.
AKS	Managed Kubernetes platform	Provided the runtime environment for internal application workloads.
kubectl	Kubernetes control tool	Used to validate nodes, pods, services, rollouts, and secret mounts.
ACR	Private container registry	Used to store and serve private container images to AKS.
Azure Arc	Hybrid-ready cluster	Connected the cluster into Azure

	management layer	Arc to demonstrate extension support and hybrid-ready operations.
Flux GitOps	Continuously applies Git state to the cluster	Made GitHub the source of truth for application delivery.
Git / GitHub	Version control and collaboration	Stored Terraform files, GitOps manifests, screenshots, and final documentation.
Azure Key Vault	Centralized secret management	Stored runtime secrets outside the cluster and outside source control.
Workload Identity	Maps Azure identity to Kubernetes service accounts	Allowed pods to authenticate to Azure securely without embedded secrets.
Secrets Store CSI Driver	Mounts external secrets into pods	Used to expose Key Vault secrets inside workloads at runtime.
Cluster Autoscaler	Scales node pool size automatically	Added elasticity so the cluster could react to workload growth.

5. Implementation Walkthrough

5.1 Infrastructure provisioning with Terraform

The project began by creating a Terraform structure for the Azure resource group and AKS cluster. This established the platform foundation as code and provided a repeatable deployment pattern. ACR was also brought under Terraform management so the private image registry became part of the controlled infrastructure state.

5.2 AKS deployment and validation

After provisioning, the cluster was accessed with Azure CLI and kubectl. The node status, namespaces, and system pods were checked to confirm that the control plane and worker nodes were healthy and that the default AKS components were running correctly.

5.3 Application exposure through LoadBalancer

A sample nginx workload was deployed into the demo namespace and exposed through a Kubernetes Service of type LoadBalancer. This step proved that AKS could host an application and expose it externally through a public endpoint.

5.4 Private image delivery through ACR

The image flow was then improved by creating an Azure Container Registry and attaching it to AKS. The nginx image was imported into ACR and the deployment was updated to pull from the private registry instead of a public image source.

5.5 Azure Arc connection

The AKS cluster was connected to Azure Arc to show that the environment could participate in hybrid-ready operations and support Arc-based extensions such as Flux.

5.6 GitOps with Flux

A GitOps repository structure was created with infrastructure, application, and cluster folders. Flux was installed as a cluster extension and configured to reconcile the manifests from GitHub. A Git-based replica change was used as proof that changes pushed to the repository were applied automatically to the cluster.

5.7 Scaling and autoscaling

A replica change created additional scheduling demand, which highlighted the need for cluster elasticity. Cluster Autoscaler was enabled on the AKS node pool with a minimum and maximum node count, allowing the platform to scale node capacity more intelligently.

5.8 Key Vault, Workload Identity, and CSI

A Key Vault secret was created and a user-assigned managed identity was configured for workload access. A federated credential linked the Kubernetes service account to the Azure identity. The secret was mounted into a test pod through the Secrets Store CSI Driver, proving that application secrets could be retrieved securely without storing them in Git or native Kubernetes Secret objects.

6. Security Model

The security design is one of the most important outcomes of the project. Instead of hardcoding credentials or copying secrets into manifests, the platform retrieves secrets at runtime from Azure Key Vault.

- Secrets remain in Azure Key Vault instead of source control.
- Pods authenticate with Workload Identity instead of embedded credentials.
- The federated credential maps the AKS service account to a managed identity in Azure.
- The Secrets Store CSI Driver mounts the secret into the pod only when the workload runs.
- The pattern supports better separation between deployment logic and sensitive runtime data.

7. GitOps Operating Model

GitOps changed the deployment model from imperative cluster changes to repository-driven reconciliation. In this project, GitHub stored the manifests, Flux watched the target path, and the cluster continuously converged toward the desired state.

The main benefit of this model is control. Changes are visible in commit history, easier to review, easier to roll back, and more consistent than direct kubectl edits made from a terminal session.

A replica update was pushed to the repository and then observed on the cluster as evidence that the Flux configuration was working correctly.

8. Validation Evidence

The following screenshots capture the major validation points from the implementation. They show the platform resources, AKS cluster state, registry configuration, Azure Arc connection, GitOps status, identity setup, and application test results.

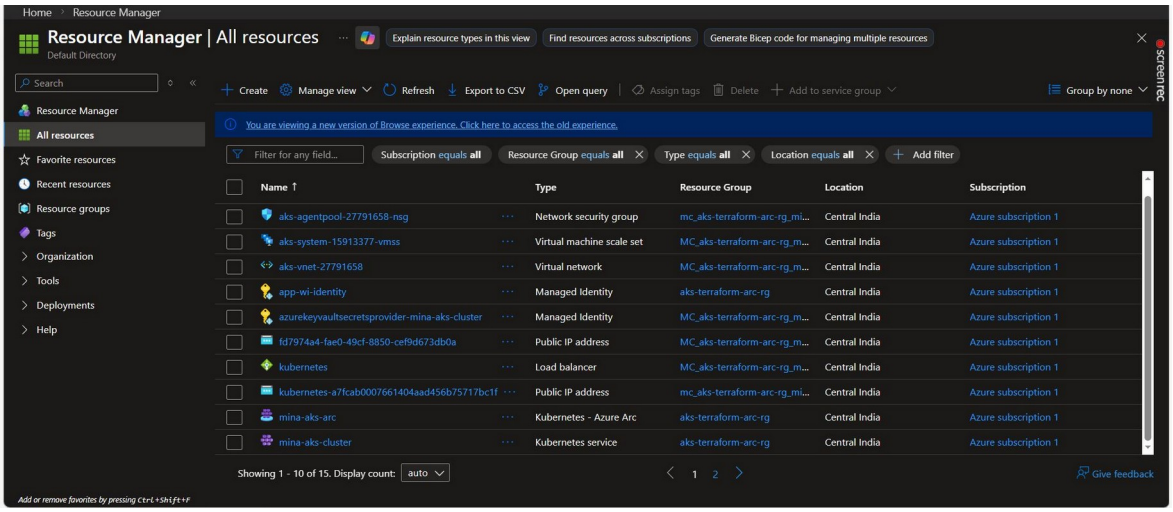

Figure 3. Azure resource overview - page 1.

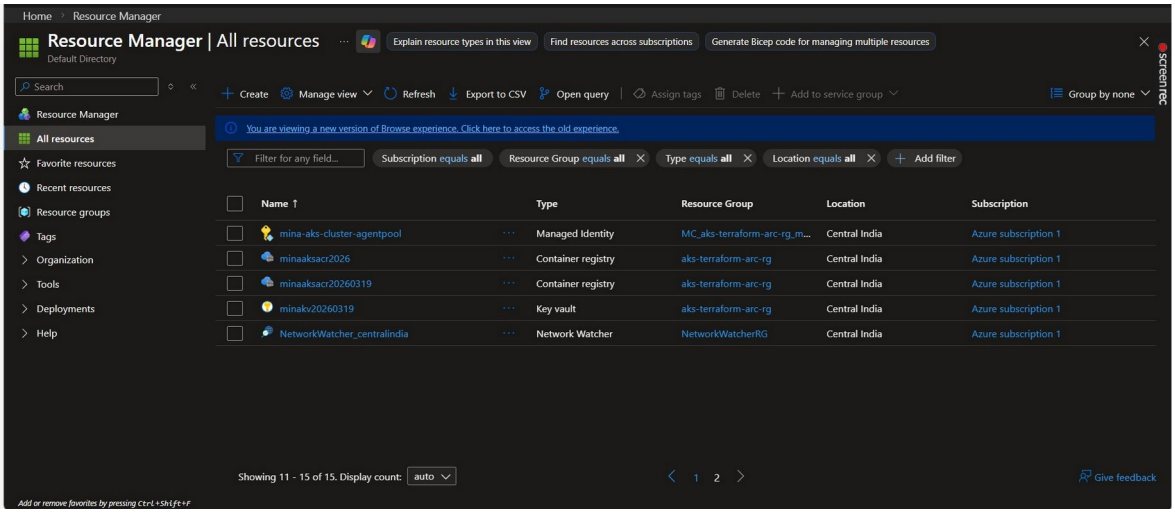

Figure 4. Azure resource overview - page 2.

```
##################################################
03 - AKS Cluster Overview
##################################################

Name               Location      KubernetesVersion    ProvisioningState    PowerState    NodeRG
-----------------  ------------  -------------------  -------------------  ------------  ------------------------------------------------------
mina-aks-cluster   centralindia  1.33                 Succeeded            Running       MC_aks-terraform-arc-rg_mina-aks-cluster_centralindia

==================================================
03 - AKS Cluster Overview
[illegible] Screenshot [illegible] [illegible] Enter...
==================================================
```

Figure 5. AKS cluster overview from PowerShell validation output.

```
##################################################
04 - AKS Node Pool / Autoscaler
##################################################

Name    VMSize           Count    EnableAutoScaling    MinCount    MaxCount    Mode
------  ---------------  -------  -------------------  ----------  ----------  ------
system  Standard_B2s_v2  2        True                 1           3           System

==================================================
```

Figure 6. AKS node pool autoscaler confirmation.

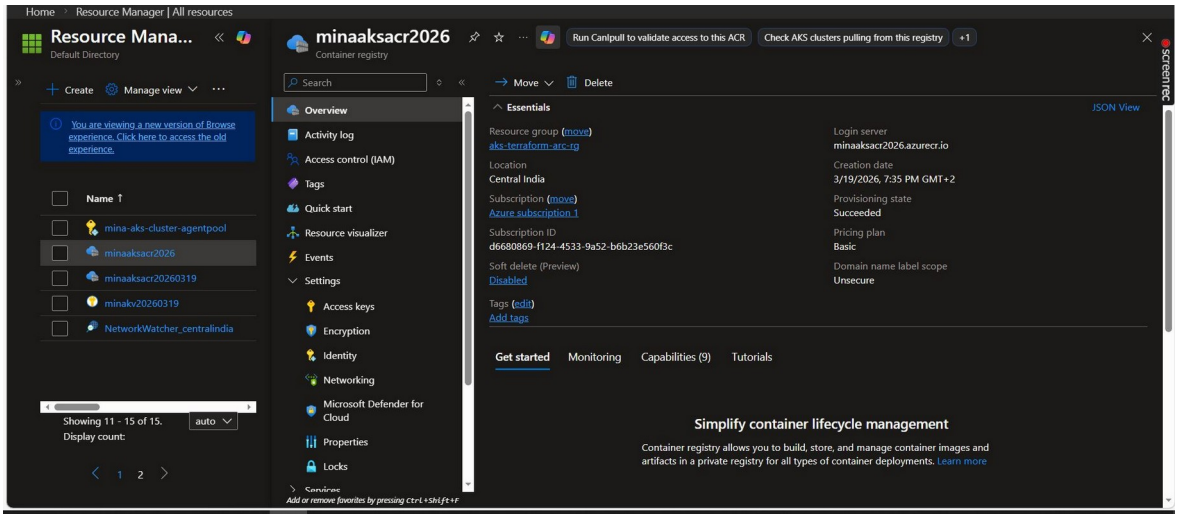

Figure 7. Azure Container Registry overview.

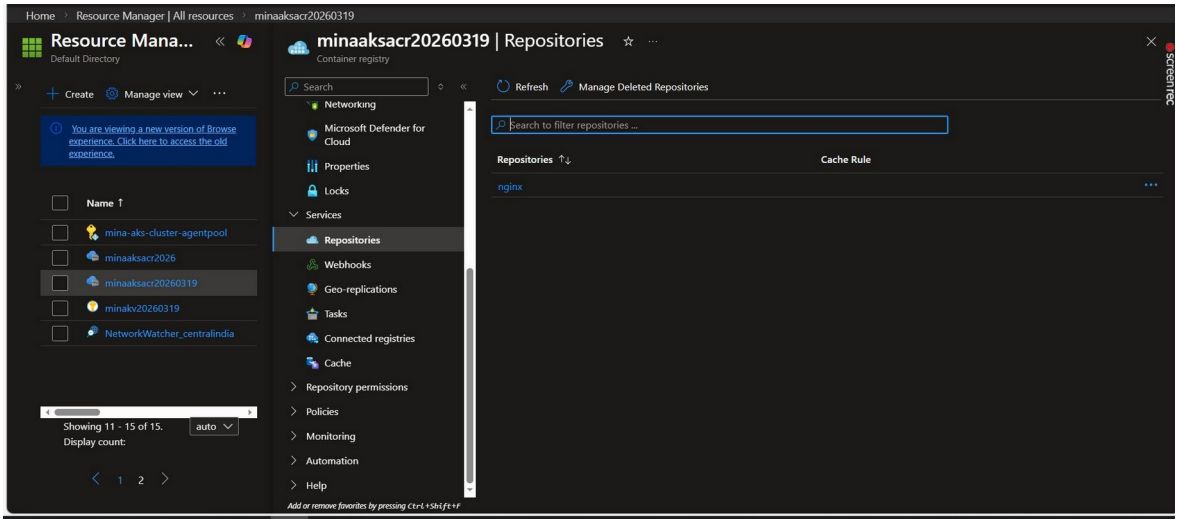

Figure 8. ACR repository and imported nginx image.

```
##################################################
11 - Azure Arc Connected Cluster
##################################################

D:\a\_work\1\s\build_scripts\windows\artifacts\cli\Lib\site-packages\cryptography/hazmat/backends/openssl/backend.py:8: UserWarning: You are using cryptography on a 32-bit Python on a 64-bit Windows Operating System. Cryptography will be
 significantly faster if you switch to using a 64-bit Python.
Name         Location     ResourceGroup
-----------  -----------  ------------------
mina-aks-arc  centralindia  aks-terraform-arc-rg
NAME                                        READY   STATUS    RESTARTS   AGE
cluster-metadata-operator-65d76b7c67-xn2c4  2/2     Running   0          3h14m
clusterconnect-agent-54c4fc6995-5nz82       3/3     Running   0          3h14m
clusteridentityoperator-6757bb896b-t4nj4    2/2     Running   0          3h14m
config-agent-84b99d8dfd-2qbs7               2/2     Running   0          3h14m
controller-manager-644df895f9-96rv2         2/2     Running   0          3h14m
extension-events-collector-8458bd4b56-sw8d8 2/2     Running   0          3h14m
extension-manager-85c9876698-f9z1t          3/3     Running   0          3h14m
flux-logs-agent-894b69cd9-fnwrt             1/1     Running   0          3h14m
kube-aad-proxy-5677f9f7d9-k91k8             2/2     Running   0          3h14m
logcollector-d6547d47b-4qtqv                1/1     Running   0          3h14m
metrics-agent-b64dddf67-hs2wf               2/2     Running   0          3h14m
resource-sync-agent-66fc77dcd5-1sd4v        2/2     Running   0          3h14m
```

Figure 9. Azure Arc connected cluster view.

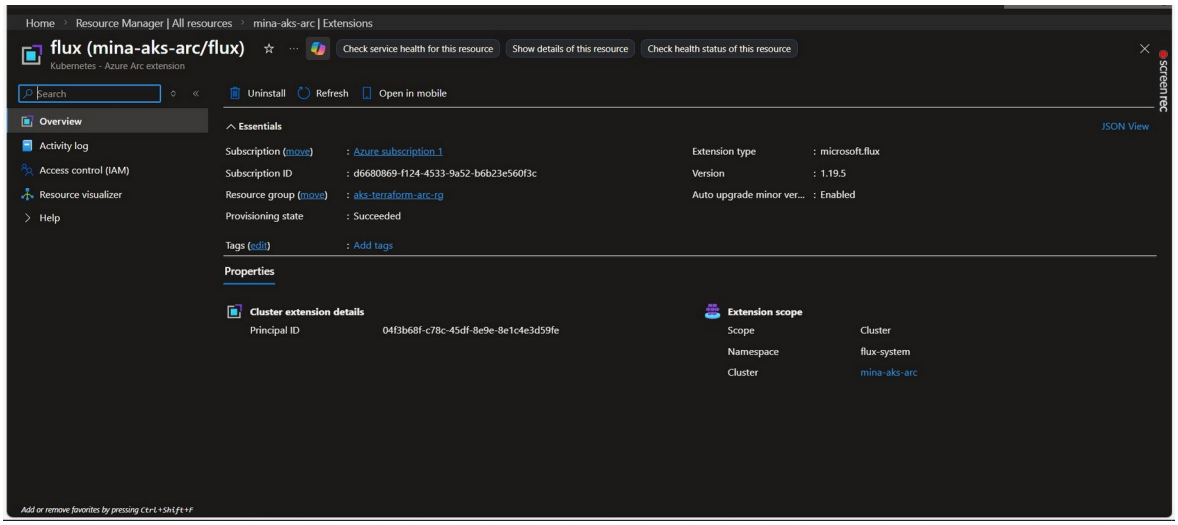

Figure 10. Flux extension overview.

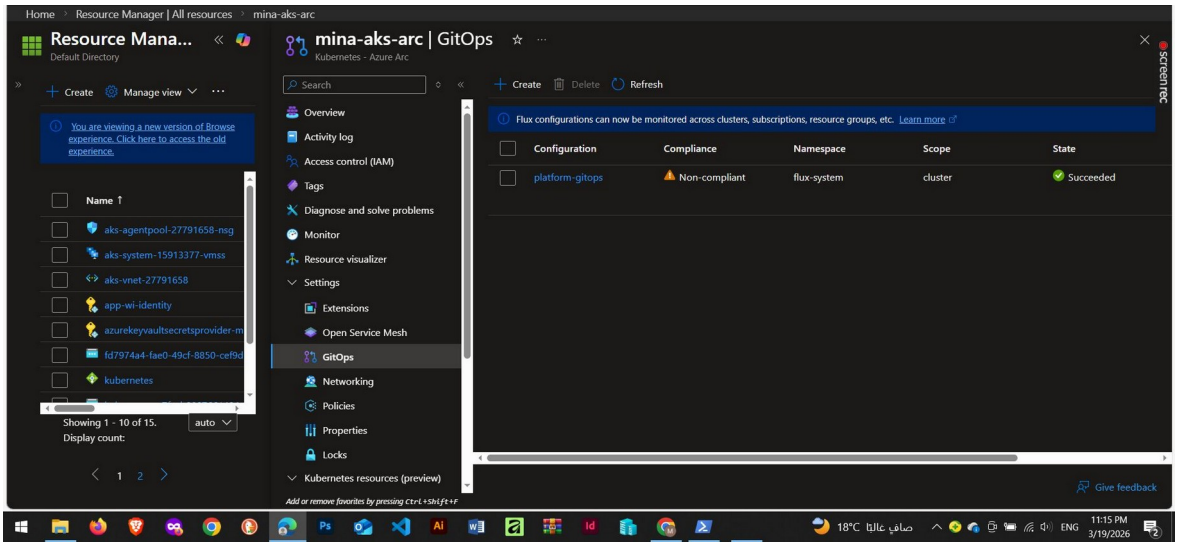

Figure 11. GitOps configuration view.

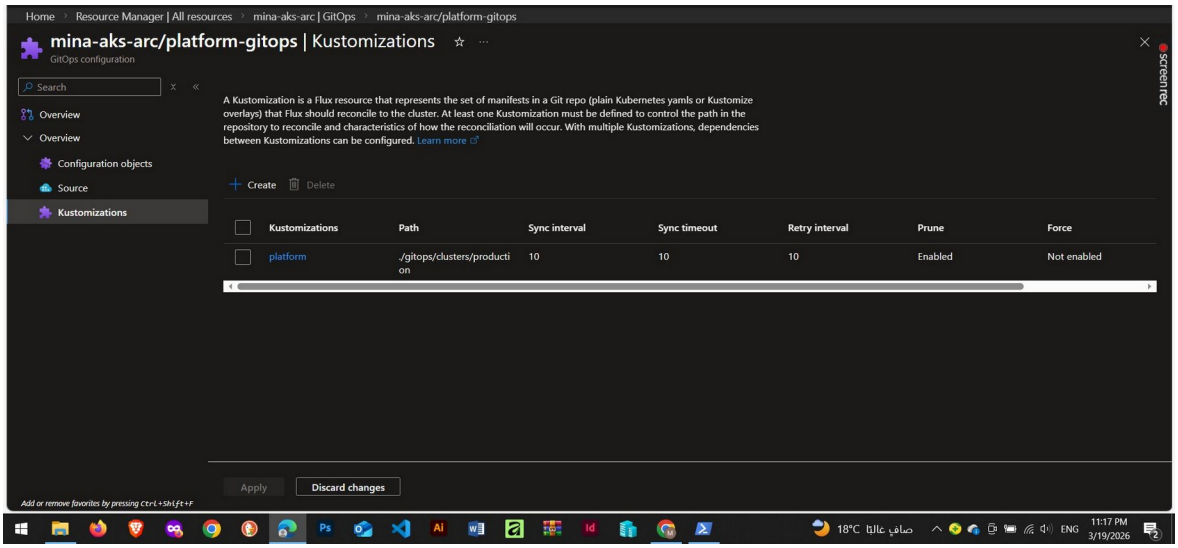

Figure 12. GitOps kustomization status.

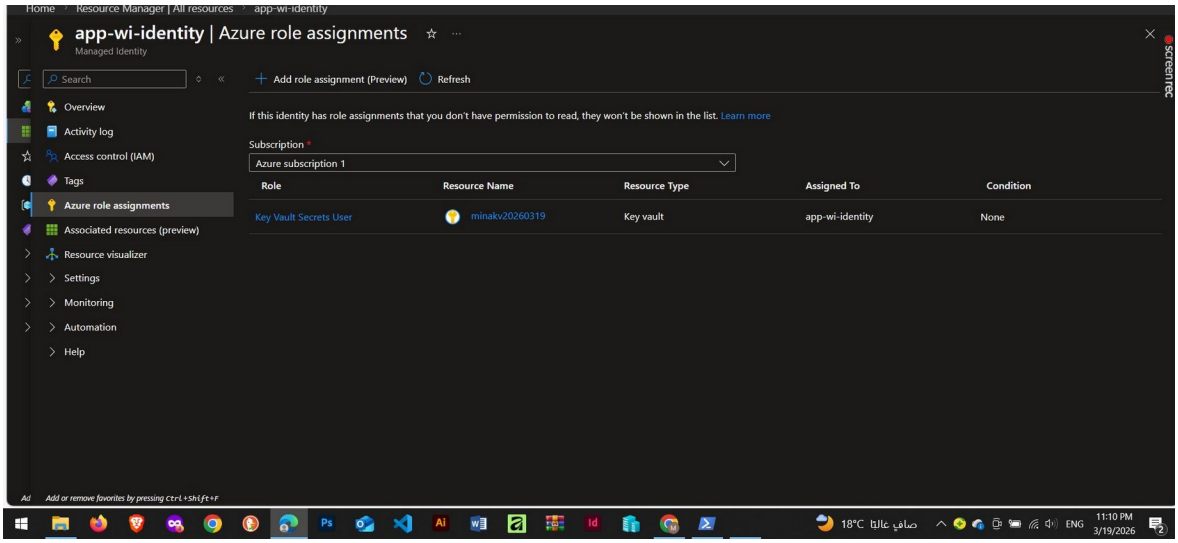

Figure 13. Managed identity role assignment.

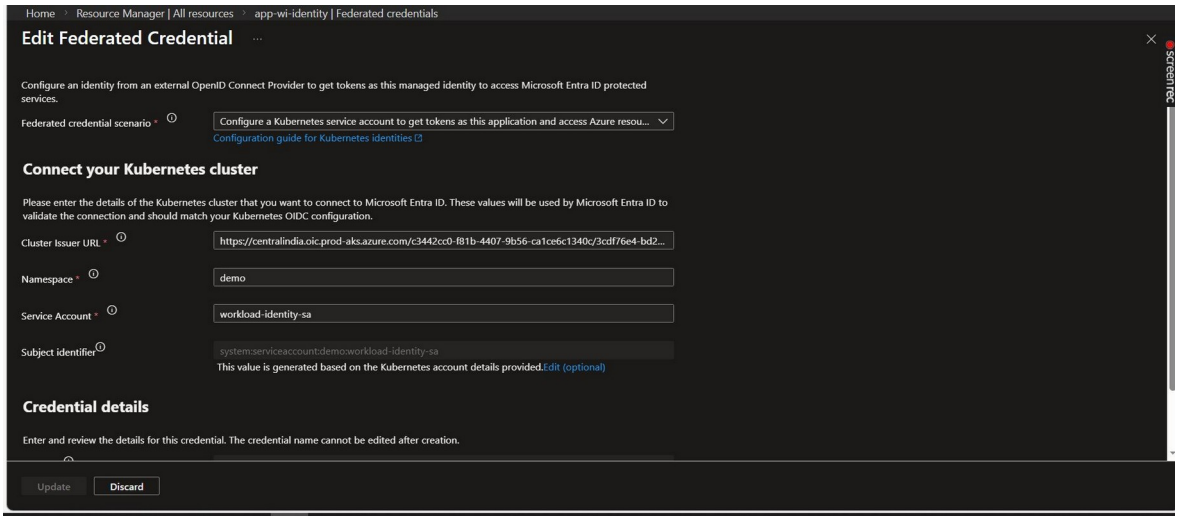

Figure 14. Federated credential configuration.

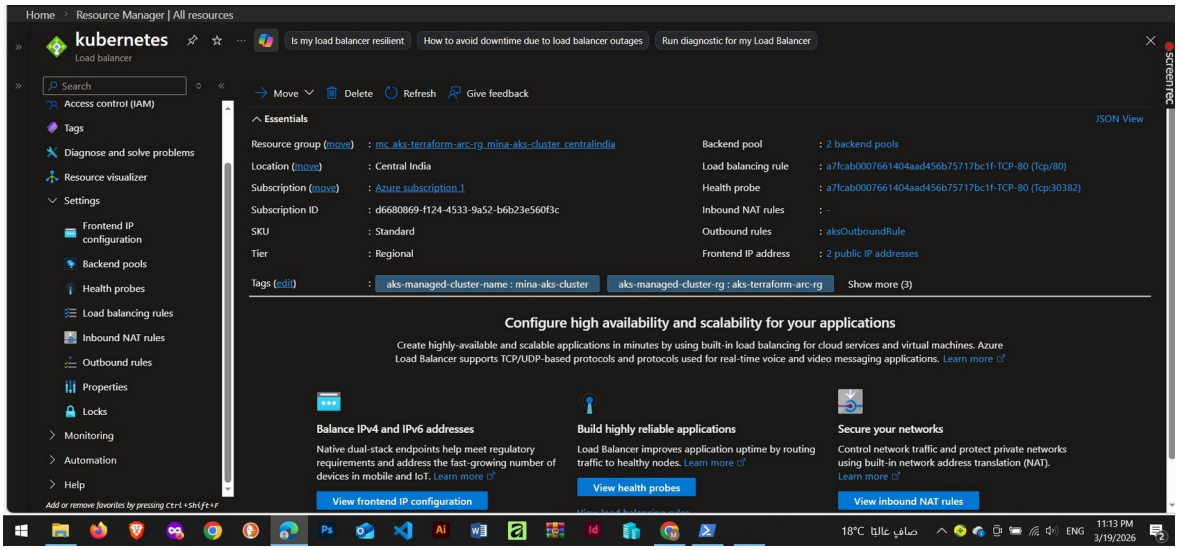

Figure 15. AKS service and application exposure.

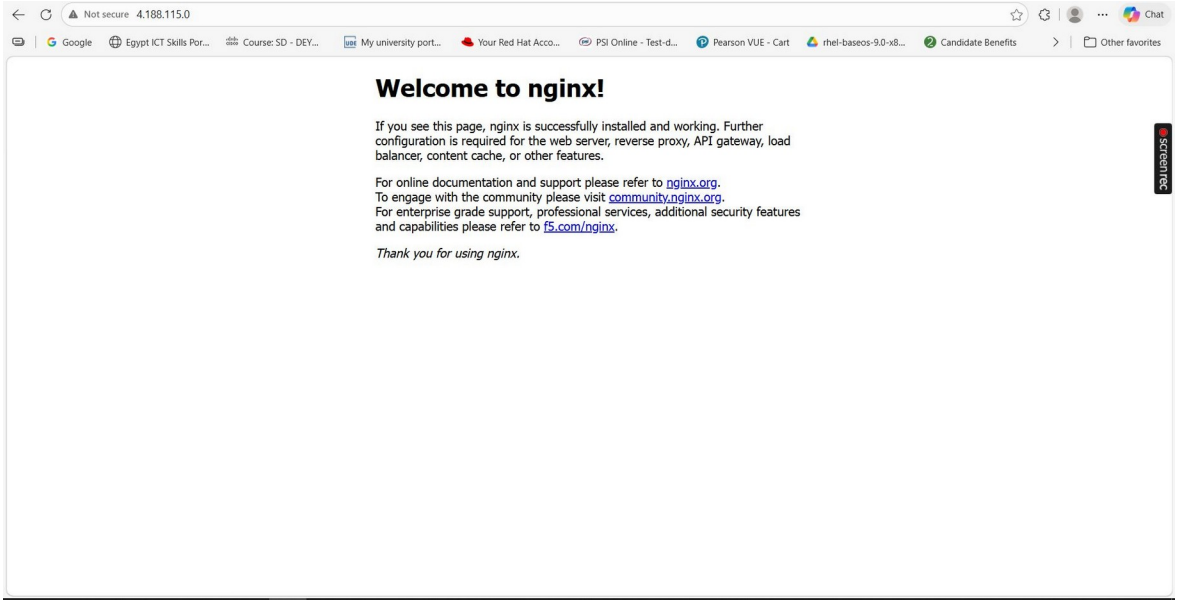

Figure 16. Browser validation of the application endpoint.

9. Project Status and Cost Note

The Azure resource group used during validation was intentionally deleted after the technical checks were completed. This was done to avoid ongoing cloud cost while keeping the repository, screenshots, and documentation as proof of the completed implementation.

Because the environment was decommissioned after validation, the repository should be understood as a documented implementation record rather than a currently live hosted environment.

10. Lessons Learned

- Terraform is effective for building a clean base platform, but importing manually created resources should be cleaned up once the final structure is stable.
- Flux turns deployment into a controlled Git-driven process and provides strong operational visibility.
- Workload Identity is a much stronger pattern than storing credentials in manifests or environment variables.
- Key Vault integration adds real platform value because it addresses an actual security problem instead of adding cloud services only for demonstration purposes.
- Cluster Autoscaler is important even in small projects because GitOps changes can quickly create more scheduling demand than a single node can support.
- Azure Arc becomes more valuable when the same pattern is extended to additional clusters outside Azure.

11. Recommended Next Improvements

- Move the remaining manually configured identity and Key Vault resources into Terraform.
- Add Azure Monitor, Managed Prometheus, and Grafana for observability.
- Enable Azure Policy and Defender for Containers for stronger governance and security coverage.
- Introduce KEDA for event-driven workload scaling.
- Extend the architecture to a second cluster so Azure Arc demonstrates a true multi-cluster or hybrid management scenario.

12. Conclusion

This project evolved from a simple AKS deployment into a secure internal application delivery platform with real engineering value. It combines Infrastructure as Code, private image delivery, GitOps, centralized secrets management, autoscaling, and hybrid-ready operations in one design.

The result is not just a collection of Azure services. It is a documented platform pattern that can be extended into a more advanced production-style environment.